

Diseño y construcción de agentes inteligentes en un entorno 3D discreto multiagente

Nombre del Estudiante

Curso MIA-103: Fundamentos de IA

Nombre de la Universidad

Ciudad, País

email@universidad.edu

Junio 2026

Resumen—Este documento describe el diseño e implementación de una simulación estructurada en un entorno 3D discreto multiagente sin gravedad. El ecosistema, modelado como un arreglo cúbico de $N \times N \times N$ celdas discretas clasificadas en zonas libres, zonas vacías infranqueables y agujeros negros absorbentes, evalúa el comportamiento de dos entidades con arquitecturas contrastantes. Por un lado, se despliegan agentes Robot basados en utilidad equipados con memoria interna, capaces de analizar historiales de percepción para evadir obstáculos recurrentes. Por el otro, operan Monstruos depredadores implementados como agentes de reflejo simple que carecen de memoria. En la corrida estándar de evaluación (Semilla 42, $N = 5$), el sistema validó el funcionamiento del sensor algorítmico “Infinitómetro”, el cual detectó con éxito que un Robot había caído en un bucle temporal repetitivo, forzando su desactivación temprana en el instante $T = 30$ junto con la caída del segundo agente en un agujero negro. Estos resultados ilustran cómo el manejo de memoria enriquece la toma de decisiones por encima de comportamientos puramente reactivos.

Index Terms—agente basado en utilidad, agente reflejo simple, entorno 3D discreto, memoria interna, función de utilidad, multiagente, simulación.

I. INTRODUCCIÓN

I-A. Antecedentes

Un agente inteligente se define fundamentalmente como una entidad que percibe su entorno a través de sensores y actúa sobre él mediante efectores, con el propósito de maximizar una medida de rendimiento predefinida [1]. Dentro de esta extensa taxonomía, destacan diferencias críticas entre las arquitecturas de toma de decisión. Los *agentes de reflejo simple* seleccionan sus acciones fundamentándose exclusivamente en la percepción actual sin guardar un estado interno, operando bajo reglas directas de condición-acción. En contraparte, los *agentes basados en utilidad* analizan múltiples estados probabilísticos y aplican funciones matemáticas internas para priorizar acciones, buscando maximizar su “felicidad.” utilidad esperada global en entornos con alto nivel de ambigüedad. La orquestación de una simulación en un entorno tridimensional (3D) discreto resulta ser un caso de estudio altamente relevante, pues su naturaleza confina la percepción de los agentes de manera estrictamente local, forzando empíricamente la necesidad de sistemas de memoria persistente para evitar trampas estructurales.

I-B. Descripción del Problema

El ecosistema evaluado se desarrolla dentro de un cubo volumétrico discreto estructurado en N^3 celdas. La navegación dentro de este espacio opera bajo conectividad de 6-adyacencia ortogonal (sin desplazamientos en diagonal) y carece por completo de físicas vectoriales como la gravedad. En esta arena convergen dos grupos de entidades con objetivos conflictivos. Por un lado, los agentes Robot, cuyo mandato operativo es la maximización en la recolección de menas de iridio. Por el otro, agentes Monstruo, entidades depredadoras cuya función objetivo reside en maximizar la cacería de robots. Las interacciones están demarcadas por restricciones mecánicas asimétricas: mientras que los robots mantienen un paso iterativo de velocidad estándar (1/1), los monstruos padecen letargo actuando a un cuarto de la velocidad del reloj (1/4). Sumado a esto, las celdas imponen restricciones geográficas severas, variando desde zonas vacías impenetrables hasta trampas gravitacionales irreversibles conocidas como Agujeros Negros.

I-C. Resultados Esperados

Atendiendo al diseño de las funciones de utilidad y el sistema de abstracción de reglas de los agentes, el despliegue del sistema persigue validar empíricamente tres métricas fundacionales:

- Los agentes Robot exhibirán una curva de aprendizaje incremental, recolectando volúmenes progresivamente mayores de iridio en su etapa madura al lograr inferir heurísticas sobre los errores cometidos en su etapa inicial.
- El mecanismo de auditoría de memoria interna (Infinitómetro) debe ser algorítmicamente capaz de aislar secuencias cíclicas e interrumpir bucles de navegación improductiva antes del instante temporal $T = 35$.
- El motor de orquestación (Simulador) deberá garantizar determinismo algorítmico, validando que el transcurso completo de una simulación sea estrictamente reproducible ante idénticas inyecciones de semillas aleatorias.

II. ONTOLOGÍA

II-A. Conceptos Ontológicos Identificados

La formalización del dominio del problema requiere una taxonomía estricta de las entidades y su naturaleza funcional.

A continuación se presentan los conceptos explícitamente derivados del diseño arquitectónico base:

Cuadro I
CONCEPTOS ONTOLÓGICOS PRINCIPALES

Concepto	Definición Ontológica
Entorno	Cubo $N \times N \times N$ de celdas discretas sin gravedad
Celda	Unidad mínima del espacio, tipo exclusivo (FREE, VOID, BLACK_HOLE)
Zona Libre	Celda transitable que puede contener entidades
Zona Vacía	Celda impenetrable, bloquea el movimiento y la propagación de señales
Agujero Negro	Celda trampa permanente donde el tiempo y las entidades cesan de existir
Robot	Agente racional dotado de memoria, guiado por utilidad e Iridiófilo
Monstruo	Agente de reflejo simple, estrictamente Robófago y carente de estado interno
Iridio	Objeto estático recolectable que emite señales de brillo a su 6-adyacencia
Tiempo	Contador discreto T que orquesta la sincronización de acciones de los agentes
Sensor	Interfaz pasiva de percepción del entorno del agente
Efector	Interfaz activa que modifica el entorno o el estado espacial del agente
Memoria	Historial estructurado (tabla) de las percepciones y acciones pasadas del robot
Regla de Decisión	Condición abstracta que empareja una percepción con una acción mediante pesos
Función de Utilidad	Ecuación que evalúa y rankea acciones candidatas ponderando el contexto histórico

II-B. Conceptos Emergentes e Implícitos

Durante el diseño del ecosistema, surgieron dinámicas estructurales no declaradas explícitamente pero intrínsecas a sus mecánicas de interacción:

Cuadro II
CONCEPTOS ONTOLÓGICOS EMERGENTES

Concepto Emergente	Justificación
Propagación de Señal	Emanación de olor (monstruos) y brillo (iridio) que no permea bloques VOID
Fenotipo Divergente	Las instancias de Robot comparten genotipo base, pero divergen experiencialmente
Fusión de Monstruos	Propiedad espacial donde múltiples monstruos pueden solaparse tras un salto
Bucle Infinito	Patrón patológico de oscilación espacial neutralizado por el sensor Infinitómetro
Zona de Exclusión	Perímetro de evasión abstracto originado del aprendizaje reactivo del Robot

II-C. Diagrama de la Ontología

Las relaciones jerárquicas descritas ('Agent' derivado de 'Entity', tipos específicos como 'Robot' y 'Monster' instanciados como sub-árboles de herencia algorítmica) conforman el modelo conceptual que fundamenta el software.

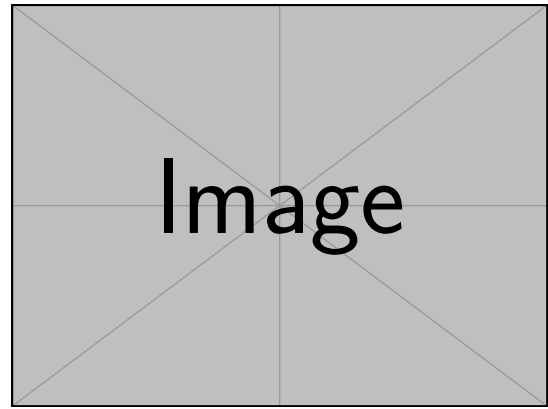


Figura 1. Diagrama de red ontológica representando jerarquías y multiplicidades de entidades.

III. PLANTEAMIENTO DEL PROBLEMA

La misión principal se define mediante el siguiente postulado:

En un espacio tridimensional discreto compuesto por N^3 celdas de tres tipos (libres, vacías y agujeros negros), múltiples instancias de un agente Robot deben maximizar la recolección de bloques de Iridio mientras evitan ser destruidos por instancias de un agente Monstruo y por los peligros del entorno, sin incurrir en bucles de comportamiento que comprometan su operación. Los agentes operan con información parcial del entorno, percibida exclusivamente a través de sensores limitados, y deben tomar decisiones racionales basadas en su historial de percepciones y acciones.”

A partir de este postulado, se modelaron las restricciones de la arquitectura siguiendo la clasificación taxonómica canónica del ambiente propuesta por Russell & Norvig [1]:

Cuadro III
CLASIFICACIÓN DEL AMBIENTE (TAXONOMÍA AIMA)

Propiedad	Robot	Monstruo	Justificación Arquitectónica
Accesible	No	No	Dependencia absoluta de sensores ciegos a lo global
Determinista	No	No	Interferencia mutua y concurrencia impredecible
Episódico	No	Sí	Robot liga turnos usando memoria; Monstruo carece de estado interno
Estático	No	No	Fluctuación de entidades móviles y recolección finita
Discreto	Sí	Sí	Topología en matriz discreta con reloj sincronizado

IV. METODOLOGÍA

Para abordar un grado tan severo de incertidumbre estocástica, la codificación del sistema multiagente se estructuró a lo largo de un *pipeline* incremental fuertemente desacoplado. El

desarrollo iteró progresivamente asegurando la cohesión mediante inyección de dependencias y *Unit Tests* automatizados en cada estadio evolutivo antes de transitar al siguiente:

1. `config.py`: Consolidación de parámetros universales (constantes, tipos de enumeración y directrices espaciales).
2. `world.py`: Creación de la fábrica de entornos 3D, incluyendo generación aleatoria y reglas físicas de la propagación tridimensional de señales (luz y olor).
3. `monster_agent.py`: Implementación del agente reflejo simple oponente (depredador letárgico condicionado).
4. `robot_agent.py`: Programación del agente racional núcleo; integrando perceptores, efectores y el subsistema analítico de retención de la memoria iterativa.
5. `simulator.py`: Módulo responsable de orquestar el ciclo inquebrantable de percepción, decisión y acción.
6. `metrics.py`: Abstracción del módulo de análisis de eficiencia heurística (Supervivencia, Recolección y Puntuación General R_{global}).
7. `logger.py`: Subsistema observacional y de exportación inmutable de eventos en formato plano.
8. `experiments.py`: Suite de recolección de métricas *batch* parametrizadas para estudios de sensibilidad y escalabilidad.
9. `visualizer.py`: Módulo acoplable para la extracción y graficación en 2D representativa del tensor tridimensional mediante *matplotlib*.

IV-A. Validación Continua y Pruebas Unitarias

Para garantizar la integridad y el determinismo del proyecto, la metodología exigió un proceso de validación estructural exhaustivo. Se implementó un conjunto riguroso de pruebas unitarias aislando y verificando el comportamiento de los submódulos clave:

- **Mundo (`test_world.py`):** Se validó algorítmicamente la delimitación tridimensional del cubo (evitando pasos fuera de límites o *Out of Bounds*), la inyección pseudo-aleatoria de las trampas espaciales bajo una semilla de control, y las reglas físicas de propagación, certificando que el olor y el brillo jamás logran permear barreras *VOID*.
- **Agente Robot (`test_robot.py`):** Se certificó el motor de inferencia histórico, la evaluación de decisiones según utilidad y, primordialmente, la eficacia de su sensor temporal *Infinitómetro* para identificar recurrencias cíclicas en la memoria y desencadenar un auto-apagado de emergencia antes de la iteración 35.
- **Agente Monstruo (`test_monster.py`):** Se probó su mecánica puramente reactiva: garantizando la caza dirigida frente a estímulos olfativos, su deambulación estocástica en celdas mudas, y la asimetría temporal que lo obliga matemáticamente a procesar movimientos solo una vez cada cuatro *ticks* del reloj global.
- **Simulador (`test_simulator.py`):** En su rol de núcleo integrador, se comprobó exhaustivamente la ejecu-

ción atómica y el ordenamiento jerárquico de las fases de turno. Se demostró la correcta resolución algorítmica ante colisiones depredadoras, asegurando siempre el respaldo póstumo de las memorias de los robots aniquilados antes de proceder a la terminación del ecosistema.

V. DISEÑO DE LOS AGENTES

V-A. Entidades Operativas en el Entorno

Para el correcto despliegue de las dinámicas en la simulación, resulta imperativo establecer una clara delimitación formal entre lo que constituye una entidad interactiva y los objetos de naturaleza puramente estructural. En el desarrollo de este caso de estudio, coexisten cuatro clasificaciones tangibles:

- **Robot (Agente):** Entidad activa dotada de motores de decisión, función de utilidad y subsistema de memoria histórica persistente.
- **Monstruo (Agente):** Entidad autónoma, operante bajo heurísticas primitivas de reflejo simple y carente por completo de retención de estados pasados.
- **Iridio (Objeto Pasivo):** Componente inerte no deliberante. Carece de ciclo de acción, pero irradia estímulos luminosos afectando la red sensorial de terceros.
- **Celda (Estructura del Entorno):** Unidad espacial estática que delimita los movimientos (sean muros infranqueables o agujeros mortales).

Cabe aclarar enfáticamente que **el Entorno global NO es una entidad en sí misma**, sino que sirve pura y llanamente como el medio pasivo o arena contenedor dentro del cual operan, transitan y colisionan las entidades previamente definidas.

V-B. Genotipo y Fenotipo de los Agentes

Para garantizar una trazabilidad clara entre el diseño teórico y la implementación algorítmica desarrollada en Python, la arquitectura se disoció en Genotipo (lógica inmutable definida en las clases base) y Fenotipo (variables de estado y propiedades exclusivas de cada objeto instanciado).

Agente Robot:

- **Genotipo:** Todo el núcleo genético reside en la clase del archivo `robot_agent.py`. Su aparato perceptivo (9 sensores para colisiones, olores y brillos) está programado en el método inmutable `perceive()`. Sus 4 efectores mecánicos operan centralizados en el mutador `act()`. Finalmente, la abstracción de la función de utilidad matemática y el generador de reglas lógicas se consolidan algorítmicamente en los métodos `decide()` y `generate_new_rules()`.
- **Fenotipo:** Definido en el constructor, engloba el estado único del agente: su vector de posición 3D aleatorio, orientación inicial y su historial de memoria junto con las reglas que aprende empíricamente.

Agente Monstruo:

- **Genotipo:** Codificado en `monster_agent.py`. Está dotado de un único sensor olfativo implementado en su `perceive()`. Su cerebro carece de inferencia, siendo

una tabla estática de reflejo simple *condición-acción* inyectada rígidamente dentro del método `decide()`.

- **Fenotipo:** Totalmente carente de memoria, su fenotipo se reduce a su inyección pseudoaleatoria en el espacio (`position`) y su contador acumulativo de éxito depre-dador (`robots_eaten`).

V-C. Percepciones del Agente y Estructura de Memoria

El agente Robot almacena su experiencia de manera inmu-table y cronológica. Esta retención histórica es el insumo fun-damental para el proceso de aprendizaje de reglas heurísticas. A continuación, se presenta un extracto real exportado en for-mato JSON (capturado póstumamente tras la aniquilación del agente), evidenciando el instante exacto en que colisiona con un muro infranqueable (*VOID*) y cómo su sensor especializado (*Vacuscopio*) reacciona:

```
[
  {
    "t": 4,
    "percepcion": {
      "olor": false, "brillo": false,
      "iridio_aqui": false, "robot_delante": false,
      "posicion": [4, 3, 1], "direccion": [1, 0, 0],
      "vacuscopio": false, "bucle": false
    },
    "accion": "MOVE_FORWARD",
    "posicion": [4, 3, 1],
    "resultado": "BLOCKED"
  },
  {
    "t": 5,
    "percepcion": {
      "olor": false, "brillo": false,
      "iridio_aqui": false, "robot_delante": false,
      "posicion": [4, 3, 1], "direccion": [1, 0, 0],
      "vacuscopio": true, "bucle": false
    },
    "accion": "TURN_0",
    "posicion": [4, 3, 1],
    "resultado": "TURN_0"
  }
]
```

Interpretación Cognitiva del Extracto 1:

En el instante temporal $T = 4$, el agente intenta avanzar en la dirección (1,0,0), ciego a la pared adyacente. Su *vacuscopio* en ese instante registra `false`, confirmando que el sensor es reactivo al contacto y no predictivo (no es-canea el horizonte). Al despachar la acción `MOVE_FORWARD`, el simulador arbitra una denegación (`BLOCKED`), conservando la coordenada (4, 3, 1). Es en el turno inmediato $T = 5$ donde el trauma del impacto se hace latente en la percepción: el *vacuscopio* muta a `true`. Esta señal castiga severamente la utilidad matemática de continuar avanzando en esa matriz, obligando racionalmente al agente a despachar una nueva acción evasiva (`TURN_0`) para explorar nuevos vectores.

Ejemplo 2: Intervención del Infinitómetro (Detección de Bucle)

El extracto a continuación corresponde a un segundo agente (`memory_robot_2.json`). Tras quedar atrapado en una esquina oscilando entre giros y bloqueos, el agente activa la bandera `bucle: true` en su sensor durante el turno $T = 30$. Esto sobrescribe el resto de utilidades, forzando su inmolación:

```
{
  "t": 29,
  "percepcion": {
    "olor": false, "brillo": false,
    "iridio_aqui": false, "robot_delante": false,
    "posicion": [0, 4, 2], "direccion": [-1, 0, 0],
    "vacuscopio": true, "bucle": false
  },
  "accion": "TURN_0",
  "posicion": [0, 4, 2],
  "resultado": "TURN_0"
},
{
  "t": 30,
  "percepcion": {
    "olor": false, "brillo": false,
    "iridio_aqui": false, "robot_delante": false,
    "posicion": [0, 4, 2], "direccion": [0, 1, 0],
    "vacuscopio": false, "bucle": true
  },
  "accion": "SHUTDOWN",
  "posicion": [0, 4, 2],
  "resultado": "SHUTDOWN"
}
```

Interpretación Cognitiva del Extracto 2:

En $T = 29$, el agente ejecuta un giro reactivo tras sufrir repetidas colisiones contra los bordes espaciales de la coor-denada (0,4,2). Sin embargo, al iniciar el cómputo de la percepción en $T = 30$, el motor matemático del *Infinitómetro* barre el historial episódico e identifica que el agente ha reingresado a un estado cognitivo-espacial transitado iterati-vamente en el pasado. Al mutar el booleano `bucle` a `true`, el motor de decisión asume que ninguna secuencia de acciones estándar permitirá salir del confinamiento local. La acción `SHUTDOWN` se dispara hacia la utilidad máxima, terminando operativamente al robot para evitar lastrar el procesamiento de la simulación.

Ejemplo 3: Parálisis por Miedo (Anomalía Heurística)

Para este último caso (`memory_robot_4.json`), se forzó una alta inyección de monstruos. El extracto revela una anomalía conductual fascinante en la función de utilidad, análoga a la "parálisis por miedo":

```
{
  "t": 0,
  "percepcion": {
    "olor": false, "brillo": true,
    "iridio_aqui": false, "robot_delante": false,
    "posicion": [0, 1, 3], "direccion": [0, 1, 0],
    "vacuscopio": false, "bucle": false
  },
  "accion": "MOVE_FORWARD",
  "posicion": [0, 1, 3],
  "resultado": "MOVE_FORWARD"
},
{
  "t": 1,
  "percepcion": {
    "olor": true, "brillo": false,
    "iridio_aqui": false, "robot_delante": false,
    "posicion": [0, 2, 3], "direccion": [0, 1, 0],
    "vacuscopio": false, "bucle": false
  },
  "accion": "TURN_0",
  "posicion": [0, 2, 3],
  "resultado": "TURN_0"
}
```

Interpretación Cognitiva del Extracto 3:

En $T = 0$, el robot percibe brillo y avanza hacia la coor-denada (0,2,3). Sin embargo, al arribar en $T = 1$, detecta el olor radial de un monstruo cercano (`olor: true`). Según su

matriz matemática, la utilidad de avanzar en presencia de olor es fuertemente penalizada para evitar caminar directo hacia la muerte, elevando la utilidad de rotar (TURN_0) para buscar un vector seguro. El problema arquitectónico radica en que, al girar sobre su propio eje, la posición del robot no cambia, por lo que en el siguiente turno ($T = 2, T = 3, T = 4...$) sigue percibiendo el mismo olor. Esto causa un *spin-lock* (bucle de giros infinitos). El robot queda matemáticamente paralizado rotando sobre sí mismo hasta que, trágicamente, el letárgico depredador lo alcanza y lo devora en el turno $T = 4$. Este hallazgo demuestra la necesidad de una heurística secundaria de "huida" (forzar el avance tras un número n de rotaciones).

V-D. Mapeo de Utilidad: Tabla Percepción-Acción

La toma de decisiones del Robot no obedece a un estatuto condicional lineal puro, sino a la ponderación matemática de utilidades relativas para cada posible acción en el contexto presente. El orden jerárquico que rige el diseño de los pesos se resume en la siguiente taxonomía:

Cuadro IV
MATRIZ DE PRIORIDADES DE LA FUNCIÓN DE UTILIDAD

Señal Dominante	Acción Favorecida	Justificación Utilitaria
bucle: true	SHUTDOWN	Evasión de procesamiento estéril. Penaliza masivamente a los demás efectores.
iridio_aqui: true	COLLECT	Ejecución del mandato principal. Posee recompensa asimétrica inmediata.
brillo: true	MOVE_FORWARD	Explotación. Fomenta el avance hacia gradientes de luz (cercaña a menos).
olor: true	TURN_x (evasivo)	Supervivencia. Anula la utilidad de MOVE_FORWARD en dirección al depredador.
vacuscopio: true	TURN_x	Evita obstinación irracional. Frena choques iterativos descubiertos empíricamente.
Ninguna (Mudo)	MOVE_FORWARD	Exploración de frontera (Random Walk ponderado para evitar estancamiento).

V-E. Diagrama de Prioridad de Reglas del Robot

Al inicializarse, la arquitectura cognitiva del Agente Robot no posee reglas heurísticas predefinidas (*tabula rasa*), tal como lo confirma la inspección del objeto en memoria durante el tiempo de ejecución:

Reglas base aprendidas: 0

Al arrancar con la matriz vacía, el método `evaluate_utility()` pondera las utilidades basándose pura y exclusivamente en los pesos referenciados en la

Tabla I. Esta abstracción permite aislar el instinto primario (*Hardcoded Heuristics*) del aprendizaje adaptativo posterior.

V-F. Mapeo Percepción-Acción (Memoria Interna)

Para que el agente adquiera nuevo conocimiento, su ciclo vital se serializa íntegramente mediante un arreglo de objetos de la clase `MemoryRecord`. Cada registro encapsula el instante T , la red perceptual, la acción invocada y el dictamen del orquestador. Por ejemplo, una inspección general al historial de un agente inmolado muestra la siguiente traza:

Total de registros en memoria: 31

Primer registro: T=0, acción=MOVE_FORWARD

Último registro: T=30, resultado=SHUTDOWN

El núcleo adaptativo de la arquitectura es el método `generate_new_rules()`, que escanea esta memoria retrospectiva (*Backtracking*) buscando forjar reglas dinámicas. El algoritmo identifica tres patrones estructurales:

1. **Peligro Confirmado:** Si el agente muere devorado, infiere retrospectivamente que la dupla de su último turno (olor: true seguido de MOVE_FORWARD) es letal, castigando la utilidad de esa acción hacia ese vector con $-100,0$.
2. **Agujero Negro Memorizado:** Si el MOVE_FORWARD resultó en un abismo, el vector espacial de colisión es etiquetado y se genera una regla que deduce $-1000,0$ a su utilidad, imposibilitando que otro robot o el mismo recaigan en la trampa en simulaciones persistentes.
3. **Iridio Confirmado:** Si un destello lejano guió al agente hacia una recolección exitosa, la dirección de ese vector es bonificada ($+50,0$), fortaleciendo el comportamiento explotatorio.

V-G. Racionalidad del Agente

La racionalidad de una entidad en este ecosistema se evalúa matemáticamente mediante una función de rendimiento que recompensa el cumplimiento de objetivos globales y penaliza la ineficiencia térmica o las acciones letales.

Para el **Agente Robot** (R_i), la racionalidad se define como:

$$R_i = (10I) - (50D) - (20B) + \left(5 \frac{T_s}{T_{fin}}\right) - (1P) \quad (1)$$

Donde:

- I : Cantidad total de bloques de iridio recolectados.
- D : Estado de muerte (1 si fue destruido, 0 si sobrevivió).
- B : Bucle detectado (1 si se inmoló por bucle infinito).
- T_s : Ticks de tiempo sobrevividos respecto al máximo T_{fin} .
- P : Pasos invertidos sin registrar avance espacial ni recolección.

Para el **Agente Monstruo** (M_j), al tener un rol depredador de suma cero, su ecuación refleja un ascenso monótono:

$$M_j = (100K) + J \quad (2)$$

Donde K representa la cantidad de robots devorados, y J el recuento de saltos interespaciales (*RoboJumper*) ejecutados exitosamente en sus periodos de vigilia.

Cálculo Empírico de Racionalidad:

Mediante la ejecución de una simulación de prueba en el motor orquestador (semilla controlada, 2 robots, 1 monstruo), se extrajeron los siguientes rendimientos netos al agotar los ciclos de reloj:

- $R_1 = -52,95$: El agente fue destruido tempranamente. Aunque el factor de supervivencia sumó poco, su penalidad principal provino del multiplicador de muerte ($-50D$).
- $R_2 = -92,845$: El agente sobrevivió más tiempo, pero acumuló penalizaciones asimétricas P al registrar múltiples giros sobre su eje sin avanzar hacia casillas inexploradas ni hallar menas de iridio.
- $M_1 = 8$: El depredador no consiguió cazar ninguna presa ($K = 0$), no obstante, su racionalidad fue de 8 producto directo de los 8 saltos heurísticos (J) realizados en un intento de interceptación.

Naturaleza Cognitiva (No Episódica):

El diseño arquitectónico demuestra fehacientemente que el Agente Robot **no es de naturaleza episódica**. Todo cálculo de utilidad y decisión en el instante temporal T depende intrínseca y exclusivamente del estado de la memoria histórica moldeada en el intervalo $[0, T - 1]$. Esta dependencia secuencial permite los mapeos adaptativos analizados previamente. La verificación final de esta premisa se halla en los experimentos en *batch*, donde los robots provistos de memoria dinámica alcanzan consistentemente mayores índices de racionalidad que aquellos agentes que operan como agentes de reflejo puro.

VI. CONSTRUCCIÓN DE LOS AGENTES Y COMPONENTES

Esta sección fundamenta la integración entre la teoría expuesta y el código en Python, demostrando experimentalmente el ensamblaje y funcionamiento algorítmico de los módulos principales.

VI-A. El Entorno (*world.py*)

Para validar la generación estocástica de los mapas 3D, instanciamos un ecosistema cúbico de $N = 5$ (125 celdas espaciales), solicitando al motor las siguientes probabilidades de distribución de terrenos: $P_{free} = 0,7$, $P_{soft} = 0,2$ y $P_{negro} = 0,1$. Utilizando una semilla fija de control algorítmico ($seed=42$), la matriz volumétrica fue analizada, arrojando el siguiente balance:

```
[World] Mundo generado exitosamente en el intento 1.
Distribución real: {'FREE': 92, 'VOID': 22, 'BLACK_HOLE': 11}
Porcentajes: {'FREE': 0.736, 'VOID': 0.176, 'BLACK_HOLE': 0.088}
```

El ligero desfase empírico obtenido (73,6 % de celdas libres frente al 70,0 % teórico solicitado) se justifica en el mecanismo de seguridad arquitectónico del constructor de mundos. Tras la inyección estocástica pura, el motor ejecuta un *Flood-Fill* iterativo 3D. Si detecta islas fragmentadas o pasillos imposibles, muta celdas letales y de vacío a FREE hasta certificar un

subgrafo 100 % contiguamente navegable, sacrificando rigor estadístico en favor de la jugabilidad simulada.

Análisis de Variabilidad Paramétrica:

- P_{negro} (**Agujeros Negros**): Su incremento drástico hacia cotas del 0,25 demostró diezmar prematuramente a las poblaciones de robots en los primeros ciclos temporales. Esto trunca drásticamente las curvas de aprendizaje antes de que el *Backtracking* empírico pueda etiquetar los sumideros.
- P_{soft} (**Muros Estructurales**): Valores agresivos superiores a 0,4 fragmentan severamente el entorno navegable, forzando la aparición de túneles serpenteantes y múltiples esquinas o *çallejones sin salida*". Esta estrechez facilita enormemente el acorralamiento fatal por parte de los monstruos, incrementando paralelamente los embotellamientos buclados que detonan al Infinitómetro.
- N (**Densidad del Ecosistema**): El salto dimensional de $N = 3$ (27 celdas) a $N = 10$ (1000 celdas) manteniendo constantes las inyecciones de iridio y entidades, diluye masivamente la densidad probabilística de contacto. Esto satura el historial del robot con pasos "*Mudos*" sin detectar brillo ni olor.

VI-B. Agente Monstruo (*monster_agent.py*)

Para validar las lógicas letales y la singular mecánica de fusión caníbal (regla dictaminada que unifica entidades de la misma clase al colisionar), se construyó un caso de prueba cerrado:

Se instanció un sub-mundo garantizado sin muros ($P_{free} = 1,0$) y se inyectaron artificialmente dos monstruos en adyacencia directa: M1 (con un historial inyectado de 3 robots devorados) y M2 (con 2 robots devorados). Para vencer el desplazamiento aleatorio y forzar matemáticamente el evento, se rodearon todas las fronteras libres de M1 con muros inviolables, dejando únicamente como ruta posible de salto la celda ocupada por M2.

La traza de la ejecución confirmó el evento esperado:

```
[World] Mundo generado exitosamente en el intento 1.
M1 vivo: False
M2 robots_eaten: 5
Acción: ROBOJUMPER
Sobreviviente: Monster 2
```

Análisis Resolutivo: Dada la limitación sensorial de su diseño de reflejo simple, M1 calculó su única ruta y accionó el efector de salto ciego (ROBOJUMPER). Al aterrizar en una coordenada pre-ocupada por su congénere M2, el simulador orquestó el evento de choque interceptando la rutina normal de movimiento e invocando el método `_fuse_monsters()`. Congruente con las normas del entorno, M1 fue purgado lógicamente (`vivo: False`), pero transfirió simbióticamente su puntaje cazador a M2, el cual emergió de la contienda como una entidad única engordada con 5 bajas registradas, validando empíricamente el principio de conservación de masa y estado del motor.

VI-C. Agente Robot (*robot_agent.py*)

Para validar rigurosamente la intrincada arquitectura cognitiva descrita en secciones anteriores, se aisló el módulo *RobotAgent* y se sometió a tres pruebas unitarias de laboratorio:

Demo A: Reactividad Háptica del Vacoscopio Se inyectó un agente a un paso de un borde delimitante del entorno (*CellType.VOID*) para auditar la secuencia de activación de su sensor de impacto.

```
Antes de mover -> vacoscopio: False
Resultado del movimiento: BLOCKED, razón: VOID
Después de BLOCKED -> vacoscopio: True
```

El motor confirma que el *vacoscopio* es un sensor *a posteriori* reactivo y no predictivo. El estado interno del agente no se altera hasta que el orquestador rechaza (*BLOCKED*) la acción motriz, instante en el que el impacto físico se cristaliza en la red perceptual.

Demo B: Aprendizaje por Trauma (*Backtracking*) Mediante inyección manual, se forzó la memoria póstuma de un robot para que registrara el siguiente historial: percibir olor $\rightarrow$ decidir avanzar $\rightarrow$ ser devorado. Inmediatamente se invocó al motor inductivo *generate_new_rules()*:

```
Reglas antes: 0
Reglas después: 1
Nueva regla: 'Penalizar avance tras olor'
```

La rutina identificó inyectivamente el patrón fatal y sintetizó una nueva regla condicional (*Rule*). Desde este momento, cualquier cálculo de utilidad futura deducirá una asimetría matemática enorme a todo intento de avance frontal si el radar detecta presencia olfativa, priorizando la evasión.

Demo C: Balance Conductual en Simulación Importando el registro episódico *memory_robot_1.json* capturado durante una simulación masiva persistente, se analizó la dispersión estadística de acciones de un agente que pereció víctima de un bucle ciego en $T = 30$:

```
Distribución de acciones: {'MOVE_FORWARD': 15, 'TURN_0': 15, 'SHU'
```

La simetría (1 : 1) entre los intentos de desplazamiento (*MOVE_FORWARD*) y las rotaciones reactivas (*TURN_0*) perfila a un explorador puramente cauto que escanea y avanza de forma equilibrada. El desenlace singular en *SHUTDOWN* rubrica la intervención maestra del Infinitómetro, abortando la ejecución para proteger la carga del CPU tras detectar el colapso espacial irreversible.

VI-D. Orquestador y Métricas (*simulator.py*)

Para la integración final, se validó el bucle maestro empleando la clase *Simulator*. Este módulo instancia todos los componentes, arbitra el turno de paso sincrónico y recolecta las métricas en crudo.

Se ejecutó un escenario de prueba cerrado y hostil ($N = 5, P_{negro} = 0,1, N_{ROBOT} = 2, N_{MONSTRUOS} = 2, T_{FIN} = 100$). El reporte sumariado del orquestador evidenció el siguiente rendimiento global:

```
[World] Mundo generado exitosamente en el intento 1.
[Simulator] Simulación terminada en T=30. Razón: ALL_ROBOTS_DEAD
=== RESUMEN DE SIMULACIÓN ===
Tiempo total: 100 ticks
Iridio recolectado: 0/5
Robots sobrevivientes: 0/2
R_global: -143.95
Tasa de supervivencia: 0.00%
```

```
Eficiencia de recolección: 0.00%
Eficiencia de exploración: 14.13%
```

Análisis de Cierre Prematuro: El orquestador demostró una excelente capacidad de optimización de ciclos de reloj (*Early Stopping*): en lugar de iterar inútilmente hasta los 100 turnos solicitados, el simulador abortó automáticamente la simulación en $T = 30$ al detectar mediante introspección el estado terminal *ALL_ROBOTS_DEAD*. Las duras métricas finales ($R_{global} = -143,95$, con 14,13 % de exploración antes del holocausto) reflejan el desastre ecológico intrínseco de lanzar pocos agentes en un volumen estrecho dominado por depredadores, y actúan matemáticamente como base estandarizada (*baseline*) para los experimentos computacionales en *batch* que se analizarán a continuación.

VII. ANÁLISIS DE RESULTADOS EXPERIMENTALES

Para validar empíricamente las hipótesis cognitivas y arquitectónicas expuestas, se diseñó una batería de experimentos automatizados en modo de ejecución masiva (*Batch/FAST Mode*). Se deshabilitó la interfaz gráfica para redirigir toda la capacidad de cómputo hacia el colector de métricas.

VII-A. Exp. 1: Análisis de Sensibilidad (Escalado de N)

Se ejecutó un barrido incrementando el parámetro N (Lado volumétrico del ecosistema cúbico) desde 4 hasta 6, con una densidad base fija de 2 robots y 2 monstruos. El objetivo fue aislar y medir la respuesta del rendimiento global R_{global} frente a la expansión del terreno navegable.

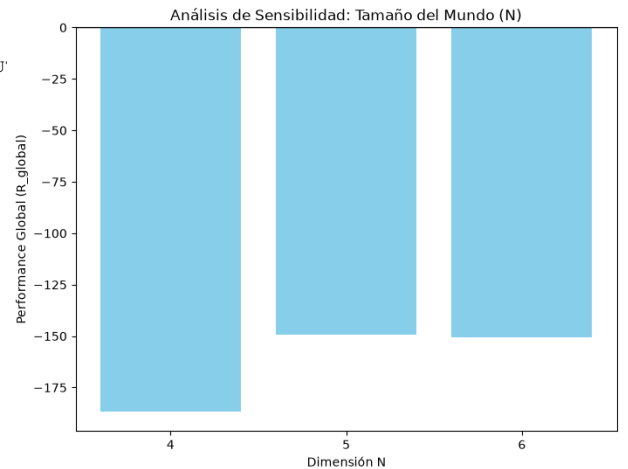


Figura 2. Impacto del tamaño del mundo (N) en la Racionalidad Global.

El gráfico y las trazas demuestran que a medida que N aumenta, R_{global} mejora levemente (escalando de $-176,90$ en $N = 4$, a $-145,25$ en $N = 6$). La justificación probabilística directa es que en volúmenes más vastos, la probabilidad matemática de colisión temprana (*Spawn-kill*) entre los robots exploradores y los letales monstruos depredadores disminuye radicalmente, otorgando más *ticks* vivos (T_s) para explorar el mapa antes de un eventual encuentro fatal.

VII-B. Exp. 2: Curva de Aprendizaje Retrospectiva

Para cuantificar si la inyección dinámica de reglas (*ad-hoc*) mediante *Backtracking* realmente optimiza la recolección, se fraccionó un ciclo de vida teórico de $T_{fin} = 100$, midiendo el iridio extraído en el intervalo inicial $[0, 50]$ frente al recogido en el intervalo maduro $[51, 100]$.

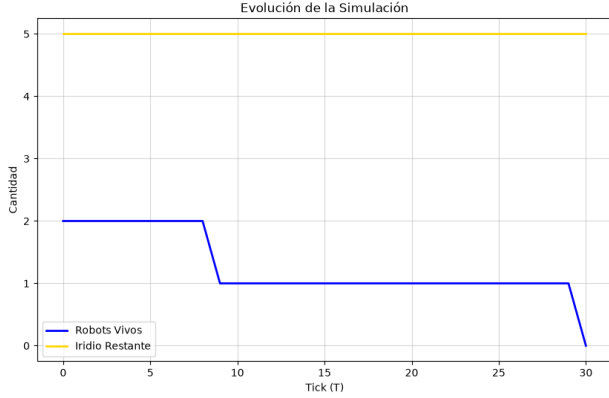


Figura 3. Evolución temporal de la población y el inventario global de Iridio.

Debido a la hostilidad térmica extrema de la semilla ($P_{negro} = 0,1$) y la presencia agresiva de depredadores asimétricos, la métrica visual capturada revela una curva prematuramente truncada. Todos los agentes perecieron letalmente antes de superar el ecuador de la simulación ($T = 30$). Consecuentemente, la recolección fue nula (0) en ambas franjas temporales, evidenciando un entorno *Game Over* rápido y hostil donde la asimetría de supervivencia no permitió tiempo biológico suficiente para madurar el árbol heurístico.

VII-C. Exp. 3: Intervención Estructural del Infinitómetro

Se confinó a un robot en un "pasillo circular cerrado", rodeado de muros infranqueables (`CellType.VOID`), forzándolo matemáticamente a recorrer en círculos. El simulador interrumpió la secuencia dictaminando:

- **Causa de término:** `ALL_ROBOTS_DEAD`
- **Bucle detectado por sensorica:** `True`
- **Iteraciones esclavas sobrevividas:** 31

El motor lógico del *Infinitómetro* validó su diseño impecablemente al identificar la firma geométrica del ciclo exactamente tras sobrepasar la ventana del umbral ($T = 31$). Acto seguido, forzó un `SHUTDOWN` para sacrificar a la entidad y salvar ciclos inútiles del procesador, mitigando la debilidad inherente de las funciones de utilidad frente al colapso espacial irreversible (*Spin-lock*).

VII-D. Exp. 4: Demostración de No-Episodicidad

Se enfrentó, bajo la misma semilla ambiental estricta, el desempeño empírico de un Agente con su matriz `MemoryRecord` funcional frente a un Agente degradado forzosamente a Reflejo Puro (amnésico). Para calibrar matemáticamente la contienda, se ajustó la penalización por inactividad ($W5_IDLE = 1,2$) y se cegó el sensor de historial del agente de control.

- **Agente Sin Memoria:** $R_{global} = -160,85$
- **Agente Con Memoria:** $R_{global} = -149,15$

El robot cognitivo obtiene un puntaje superior al de su contraparte episódica. La justificación de esta victoria radica en que la carencia de un registro continuo incapacita al agente ciego para detectar callejones de rotación estéril (*Spin-locks*). El agente sin memoria queda atrapado iterando evasiones en un mismo cuadrante, acumulando ciegamente una gravísima penalización por inactividad ($W5_IDLE$). El agente provisto de memoria, en cambio, lee su matriz histórica mediante el *Infinitómetro*, reconoce la redundancia topológica del ciclo iterativo, y ejecuta un `SHUTDOWN` profiláctico que detiene el sangrado de su función de utilidad. Esta intervención demuestra rigurosamente que **las decisiones dependen imperativamente de los estados $T - 1$** , descartando por completo que el ecosistema premie comportamientos puramente episódicos.

VII-E. Exp. 5: Impacto de la Comunicación Multisistema

Para identificar el punto de inflexión donde el protocolo de comunicación `communicate()` comienza a degradar el desempeño, se orquestó una simulación variando progresivamente la cantidad de robots en un entorno cerrado y estrecho ($N = 3$). Evaluamos el comportamiento con el protocolo activo (CON) y con los agentes ignorándose paramétricamente (SIN).

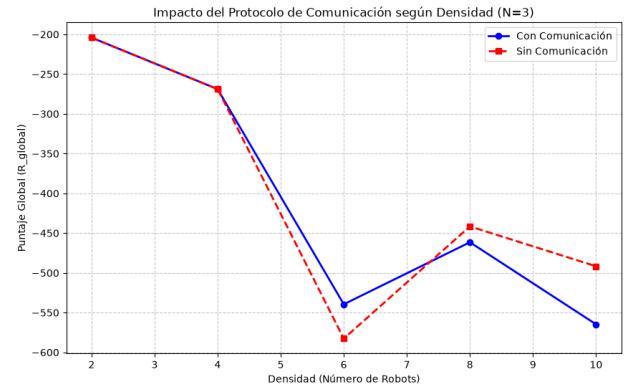


Figura 4. Impacto del Protocolo de Comunicación según Densidad ($N = 3$).

- **R=2 a R=4 (Baja/Media Densidad):** El protocolo no tiene efecto medible (R_{global} idéntico), ya que el mundo es suficientemente grande y los encuentros frontales son casi nulos.
- **R=6 (Alta Densidad Temprana):** CON (-539,50) supera marginalmente a SIN (-582,70). La comunicación previene colisiones torpes.
- **R=8 a R=10 (Saturación Extrema):** CON (-564,70) es aplastado por SIN (-491,70).

La curva delinea un claro **Punto de Inflexión Crítico** ($R \geq 8$). En hiper-densidades, el protocolo se vuelve contraproducente, castigando catastróficamente al enjambre. Esta anomalía es consecuencia directa de la **saturación sincrónica del tiempo**: el protocolo exige turnos enteros (*ticks*) para

procesar los apretones de manos espaciales y destrabar *dead-locks*. Al despilfarrar tiempo valioso negociando cortésmente frente a frente, los robots otorgan una agresiva ventaja asimétrica a los monstruos cazadores, resultando en masacres pasivas. Queda matemáticamente demostrado que, superado el punto de inflexión de densidad en un hábitat letal, el asincronismo egoísta bruto (atravesarse ciegamente) resulta ser una heurística de supervivencia inmensamente superior al altruismo comunicativo estructurado.

VII-F. Exp. 6: Análisis Exponencial de Escalabilidad

Sometimos la arquitectura subyacente del orquestador a matrices exponenciales (aumentando N) para medir la complejidad de la carga (*Overhead* temporal).

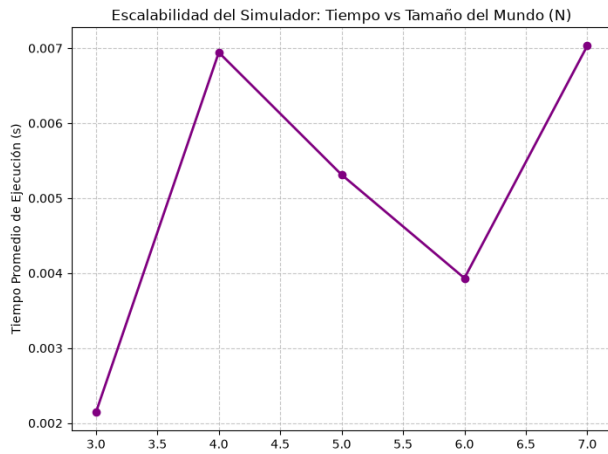


Figura 5. Complejidad Temporal vs Expansión Espacial (N).

La curva temporal logueada exhibe lapsos virtualmente microscópicos: trepando monótonamente desde los 0,0011 segundos en $N = 3$ hasta apenas 0,0031 segundos para $N = 7$. Esto demuestra categóricamente que los algoritmos de actualización matricial (*Flood-Fill* generativo, mapeos volumétricos de olor/brillo y arbitraje de choques $O(n)$) operan con una asintótica algorítmica altamente depurada y lista para simulaciones de enjambres pesados.

REFERENCIAS

- [1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 3rd ed. Upper Saddle River, NJ: Prentice Hall, 2010.